# Week 7: Advanced Airflow

Keeyong Han

# Table Of Contents


1. Interview Tips #2
2. Recap of the 5th & 6th Week
3. Incremental Update in Airflow
4. **Break: Group Project Team Gathering**
5. Useful Airflow Features
6. Window Functions
7. More on Midterms
8. No Homework & No Demo
9. Quiz #2

# Interview Tip #2

# Behavioral Questions

Figuring out your behavior from past actual events

1. Collaboration
2. Ownership
3. Impact
4. Craftsmanship

STAR (Situation, Task, Action, Result)

# Behavioral Questions: Impact

What is the most impactful project you have done so far?

- Communication skill: Understating or Overstating?
- Definition of impact?
- Lessons from the project?
- What would you do differently?

# Behavioral Questions: Collaboration

Who have been the hardest people to work with so far, and why were they difficult?

- Communication skill: being honest
  - Remember no one is perfect
- Lessons from the interaction?
- What changes have you made since?

# Recap of 5th & 6th Week

# Key Concepts

- Full Refresh, Incremental Update
- Idempotency
- SQL Transaction along with try/except
- Backfill
  - Rerunning  a data pipeline to fill incomplete or missing data
  - This makes data engineer’s job harder
- Upsert
  - Snowflake supports an operation called MERGE
  - Update existing records and insert new records
    - Most efficient way of ensuring the primary key uniqueness

# Full Refresh

```
try:
    cur.execute("BEGIN;")
    cur.execute(f"CREATE TABLE IF NOT EXISTS {target_table} ( …")
    cur.execute(f"DELETE FROM {target_table}")
    cur.execute(f"COPY INTO {target_table} …")
    cur.execute("COMMIT;")
except Exception as e:
    cur.execute("ROLLBACK;")
    raise
```

Snowflake does not support DDL operations within a transaction and always commits immediately

# Airflow @task decorators: marking a function as a task

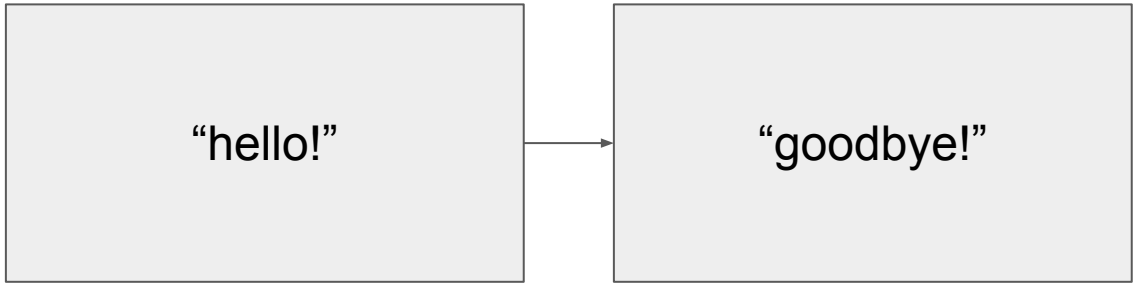
```
from airflow import DAG
from airflow.decorators import task
from datetime import timedelta
from datetime import datetime

@task
def print_hello():
    print("hello!")
    return "hello!"

@task
def print_goodbye():
    print("goodbye!")
    return "goodbye!"
```

```
with DAG(
    dag_id = 'HelloWorld',
    start_date = datetime(2024,9,25),
    catchup=False,
    tags=['example'],
    schedule = '0 2 * * *'
) as dag:

    # run two tasks in sequence
    print_hello() >> print_goodbye()
```



# Review the Country/Capital DAG

- Airflow Variables
- Airflow Connections
- GitHub link

# Incremental Update in Airflow

# Let’s say if you do an **daily** update of a stock price

- Let’s use download function of yfinance
- Everyday, get the price of the previous day
- Let’s say today is 2025-10-07

```
import yfinance as yf

data = yf.download("APPL", start="2025-10-06", end="2025-10-07")
# data is a pandas dataframe
```

Everyday, you need to know yesterday’s date and today’s date. Run the following code

data = yf.download("APPL", start=**yesterday_date**, end=**today_date**)

# How to get today’s date and yesterday’s date?

```
from datetime import datetime, timedelta

# Get today's date
today = datetime.today().strftime('%Y-%m-%d')    # strftime function converts datetime type to string

# Get yesterday's date
yesterday = (datetime.today() - timedelta(days=1)).strftime('%Y-%m-%d')

print("Today's Date:", today)
print("Yesterday's Date:", yesterday)

data = yf.download("APPL", start=yesterday, end=today)
```

# Is that enough?

What if we learned this run didn't go well later?

- What's the problem of the approach?
  - What if you have to go back and rerun any past runs? (Failed DAG Run)
- Let's say there is a pipeline running daily at 2:30AM
  - When it runs on 01/02, the data to fetch is from 01/01
  - …
  - When it runs on 03/03, the data to fetch is from 03/02
  - When it runs on 03/04, the data to fetch is from 03/03

DAG & DAG Run

**Backfill** will be a problem

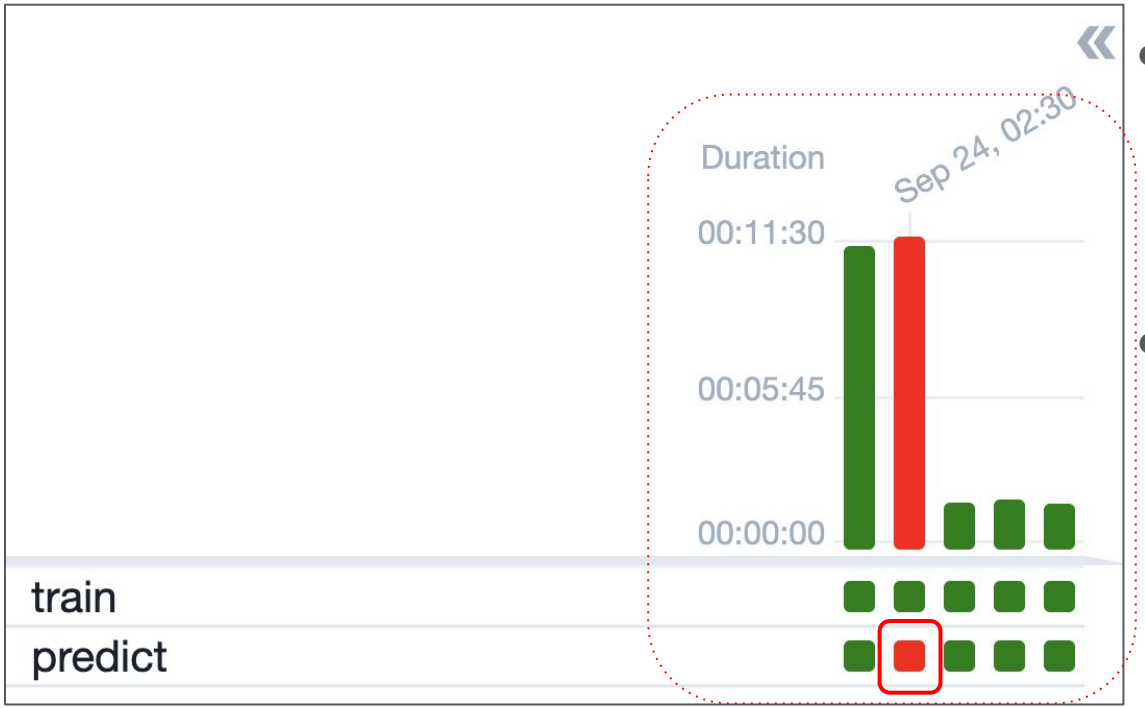
- the logic is built to get the dates **at the time of data pipeline's run (DAG Run)**
  - We need to record the dates of each data pipeline's run somewhere
- How to implement this mechanism?

# How Airflow can be helpful

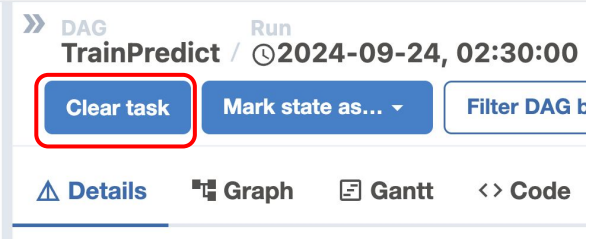
- For each DAG run, Airflow records its run statuses and associated timestamps
  - The **meta database** is used to store this information
  - There is a table called "dag_run" in the database
- Let's say there is a DAG running daily at 2:30AM
  - When it runs on 10/03, the data to fetch is from 10/02
  - When it runs on 10/04, the data to fetch is from 10/03
  - …

Airflow has a variable called "**logical_date**" which associates each DAG run with the data date to fetch

# Airflow's Approach




- You don't compute the date or the hour of data to fetch yourself but use whatever Airflow tells you ("logical_date")
- If there is any failed run, it will appear as red in the Web UI and you can rerun it




The results are stored in dag_run table

# How to get the logical_date in Airflow

from airflow.operators.python import **get_current_context**

```
def get_logical_date():
    # Get the current context
    context = get_current_context()
    # Get the logical_date as a string from the context
    logical_date = str(context['logical_date'])
    # Return the first 10 characters which is “YYYY-MM-DD”
    return logical_date[:10]
```

```
context['ds']
context['logical_date']
…
```

# How to Specify when you run Airflow commands

- “airflow dags test” or “airflow tasks test” take the logical_date as the last param
  - airflow dags test YfinanceToSnowflake_Inc 2025-10-06

logical_date

# How Airflow can get you: **start_date**

- Airflow assumes that a DAG is doing incremental update
- DAG's **start_date** refers to the date of data to copy for the first time
- If it is '2025-01-01' for a daily DAG, then the first time the DAG runs is:
  - **2025-01-02**

# Quiz #1 & #2

If a DAG has schedule of “30 2 * * *” and start_date of “2025-10-01”

1. When would it run for the first time?
   a. 2025-10-02
2. If I enable this DAG on 10/06, what will happen to the missing runs (from 2025-10-01 to 2023-10-05)?
   a. **Catchup** will determine! catchup = True
   b. Its default value is **True**. If it is True, it will try to catch up any missing runs literally

# Quiz #3

3. When a DAG has schedule of “30 2 * * *” and it runs on 10/05, what will be the values of the following variables?

| | |
|---|---|
| context['logical_date'] | 2025-10-04 02:30:00 |
| context['ds'] | 2025-10-04 |
| … | |

,

# Stock Price DAG (Incremental Update)

- Again full refresh makes thing a lot simpler
  - Stay with full refresh as long as you can
- Code:
  - Naive Implementation: GitHub link
  - Recommended Implementation: GitHub link
    - **catchUp** parameter in DAG

# Break: Group Project Team Gathering & Project Details

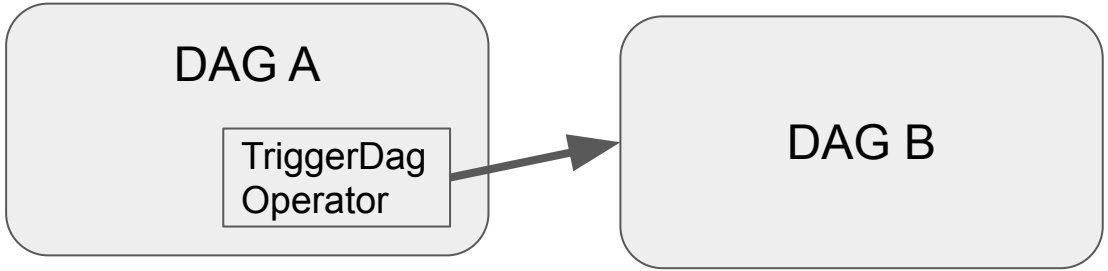
# Useful Airflow Features

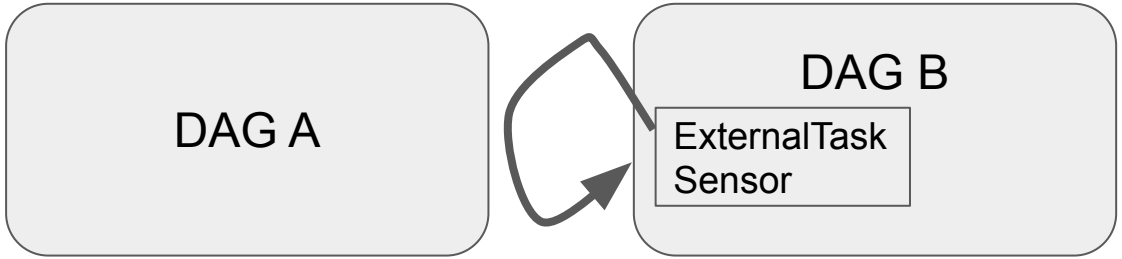
# Other Key Features of Airflow

- **How to define DAG dependencies**
  - TriggerDagOperator
  - ExternalTaskSensor
- How to integrate with Slack (for error/alert notifications)
- How to run different tasks depending on a condition
  - BranchPythonOperator
- Task Groups, API Access, …

# How to define DAG dependencies

- Explicit trigger
  - **TriggerDagOperator**
  - DAG A triggers DAG B
- Reactive trigger
  - **ExternalTaskSensor**
  - DAG B waits for a task of DAG A to complete in some fashion



# SQL Window Functions

# WINDOW Functions

- Unlike GROUP BY aggregation functions, Window functions don’t collapse rows. In other words, the number of records doesn’t change
- Syntax:
  - Window_functions OVER (PARTITION BY … ORDER BY …)
- Will learn a few functions
  - **ROW_NUMBER**
  - **LAG**
  - FIRST_VALUE/LAST_VALUE
  - SUM

# WINDOW Functions - ROW_NUMBER (1)

| userid | ts | channel |
|---|---|---|
| 10 | 2021-01-01 | google |
| 11 | 2021-01-03 | facebook |
| 11 | 2021-01-01 | naver |
| 10 | 2021-01-02 | facebook |
| 11 | 2021-01-04 | google |
| 10 | 2021-01-03 | youtube |

| userid | ts | channel | nn |
|---|---|---|---|
| **10** | 2021-01-01 | google | 1 |
| **10** | 2021-01-02 | facebook | 2 |
| **10** | 2021-01-03 | youtube | 3 |
| **11** | 2021-01-01 | naver | 1 |
| **11** | 2021-01-03 | facebook | 2 |
| **11** | 2021-01-04 | google | 3 |

3. ROW_NUMBER can be used to implement

1. What if you want to assign a sequential number per user based on the timestamp?

2. Add a new column!
Partition records by user, sort them by time within each group, and assign numbers starting from 1.

# WINDOW Functions - ROW_NUMBER (2)

- Assigns a unique sequential number to each row within a result set defined by **ORDER BY** and **PARTITION BY**
- What if you want to assign a sequential number per user based on the timestamp?

```
SELECT usc.userid, usc.channel, ROW_NUMBER() OVER (PARTITION BY usc.userid ORDER BY st.ts) nn
FROM raw_data.user_session_channel usc
JOIN raw_data.session_timestamp st ON usc.sessionid = st.sessionid;
```

# WINDOW Functions - LAG Function (1)

- When all sessions are sorted in ascending order by time for each user
  - What is the channel of the next session?
  - What is the channel of the previous session?

| userId | sessionId | channel | ts | previous channel |
|---|---|---|---|---|
| 27 | a67c8c9a961b4182688768dd9ba015fe | Youtube | 2019-05-01 17:04:00 | |
| 27 | b04c387c8384ca083a71b8da516f65f6 | Google | 2019-05-02 19:21:30 | Youtube |
| 27 | abebb7c39f4b5e46bbcfab2b565ef32b | Naver | 2019-05-03 20:38:41 | Google |
| 27 | ab49ef78e2877bfd2c2bfa738e459bf0 | Facebook | 2019-05-04 21:48:07 | Naver |
| 27 | f740c8d9c193f16d8a07d3a8a751d13f | Facebook | 2019-05-05 18:15:31 | Facebook |

```
LAG(channel,1) OVER (PARTITION BY userId ORDER BY ts) prev_channel
```

# WINDOW Functions - LAG Function (2)

```
-- Find the previous channel
SELECT usc.*, st.ts,
    LAG(channel, 1) OVER (PARTITION BY userId ORDER BY ts) prev_channel
FROM raw_data.user_session_channel usc
JOIN raw_data.session_timestamp st ON usc.sessionid = st.sessionid
ORDER BY usc.userid, st.ts
```

| userId | sessionId | channel | ts | previous channel |
|---|---|---|---|---|
| 27 | a67c8c9a961b4182688768dd9ba015fe | Youtube | 2019-05-01 17:04:00 | |
| 27 | b04c387c8384ca083a71b8da516f65f6 | Google | 2019-05-02 19:21:30 | Youtube |
| 27 | abebb7c39f4b5e46bbcfab2b565ef32b | Naver | 2019-05-03 20:38:41 | Google |
| 27 | ab49ef78e2877bfd2c2bfa738e459bf0 | Facebook | 2019-05-04 21:48:07 | Naver |
| 27 | f740c8d9c193f16d8a07d3a8a751d13f | Facebook | 2019-05-05 18:15:31 | Facebook |

# SQL Practice #4

- NULLIF, COALESCE
- UNION, EXCEPT, INTERSECT
- WINDOW FUNCTIONS
  - ROW_NUMBER, LAG, FIRST_VALUE/LAST_VALUE, …

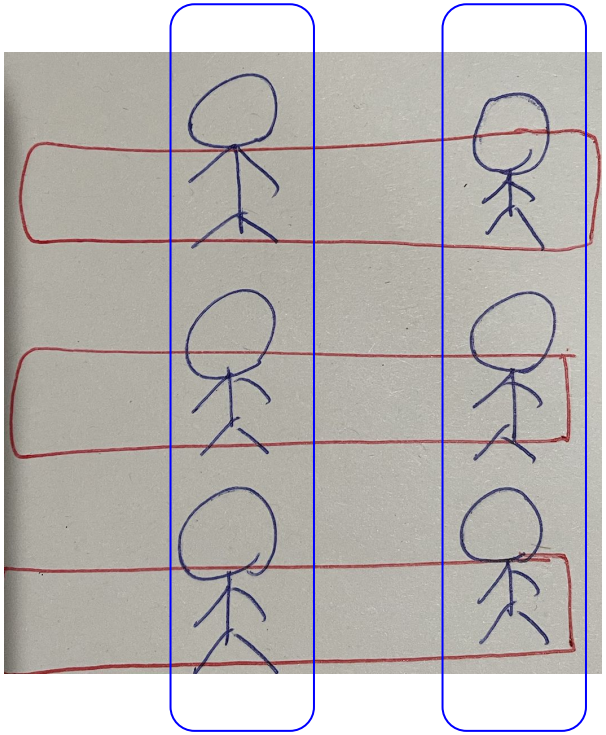
# More on Midterms

# Details of Midterms

- Covers week 1 to week 8
- Part A:
  - 45 minutes & 30 questions (60 pts)
  - Multi-choice questions
- Part B:
  - 75 minutes & 10 questions (85 pts)
  - SQL queries: https://leetcode.com/studyplan/top-sql-50/
    - 1 question from Easy SELECT will be modified
    - 1 question from Easy GROUP BY will be modified
    - 1 question from Easy JOIN will be modified
  - Concept assessment
    - Short-answer & long-answer questions

# Next Week
# ELT Overview

# Quiz #2

# Ground Rules

- 15 questions in 15 minutes
- No bathroom breaks allowed
- Put your digital devices in your own bags.
- Keep your bags on the floor
- Seating Criteria
  - Aligned in the same vertical columns.
  - Seats must be alternated within each row, with no students sitting directly beside each other.